

PO Long Text New Persistency — Skill Architecture Guide

PO Long Text New Persistency — Skill Architecture Guide

Skill Name: po-longtext-persistency-expert **Location:**
C:\Users\I308878\PO_LongText_Persistency **Date:** 2026-06-19

Executive Summary

This skill enables Claude to act as a senior SAP ABAP expert for the **PO Long Text New Persistency** project — migrating Purchase Order long texts from SAP’s compressed classic framework (STXH/STXL) to a new flat, SQL-readable custom table (ZPO_LONGTEXT) on the same SAP system.

The Problem in One Line

SAP stores PO notes in compressed tables (STXL) that cannot be read by plain SQL, CDS views, or BDC programs without expensive decompression on every access.

The Solution in One Line

Create a flat custom table ZPO_LONGTEXT on the same system — populated once via a migration program and kept in sync via a dual-write BAdI on every PO save.

Skill Folder Structure — Executive Overview

PO_LongText_Persistency/

|

— Claude.md

|

tools,

|

THE SKILL BRAIN
Full skill definition – triggers,

architecture, ABAP code, rules, quick

ref	
├─ repos.yaml	SOURCE REGISTRY Lists all raw knowledge documents
├─ Knowledge/	RAW SOURCE FILES Original handover docs, SDD, VTT
transcripts	Received from the product team – not
edited	
├─ agent/	STRUCTURED KNOWLEDGE FOR CLAUDE
│ └─ knowledge/	WHAT CLAUDE KNOWS
│ │ └─ architecture/	HOW the system works
│ │ └─ domain/	WHY it exists + business rules
│ │ └─ product/	WHAT the product spec says (future)
│ └─ codebase-index/	HOW TO WORK IN THE CODEBASE
│ │ └─ codebase-overview.md	WHAT objects exist
│ │ └─ business-rules.md	WHAT rules must never be broken
│ │ └─ data-flows.md	HOW data moves step by step
│ └─ artifacts/	GENERATED OUTPUTS (future) ABAP programs, test results
└─ PO_LongText_Skill_Reference.pdf	REFERENCE GUIDE (this document)

File-by-File Responsibility

Claude.md — The Skill Brain

One liner: The master file that defines the entire skill — Claude reads this to become a PO Long Text expert in every conversation.

Contains: - Skill name, trigger keywords, allowed tools - Business context — why the project exists - Complete architecture (single system, no Hub) - New persistence table ZPO_LONGTEXT design - Text object types (EKKO/F01, EKPO/F09 etc.) - Full ABAP code — ZCL_PO_LONGTEXT_HANDLER, ZCL_PO_TEXT_BADI_IMPL, ZPO_LONGTEXT_MIGRATE - Agent behavior guidelines — how to classify and debug tasks - Quick reference card

When used: Every time this skill is invoked. Claude reads this first.

repos.yaml — Source Document Registry

One liner: A manifest file that tells the agent framework which raw source documents exist and where to find them.

Contains: Paths to all Knowledge folder documents — pptx, docx, vtt files.

When used: By the agent framework to locate and index source documents. When you add a new document to the Knowledge folder, register it here.

Knowledge/ — Raw Source Files

One liner: The original, unmodified documents received from the product team — the ground truth that the structured knowledge files are derived from.

```
Knowledge/
├── PO Notes New Persistency/
│   ├── PO Notes to New Persistency .pptx          ← Handover slide deck
│   └── Purchase Order Notes - New persistency      ← Meeting transcript (VTT)
│       └── - Handover - Lift & Shift team (1).vtt
└── PO Notes SD implemented - Further S4 PR support required for S0 topics/
    ├── SDD New Persistency for Long Texts          ← System Design Document
    │   └── current working version.docx
    └── FW_ Further S_4 PR support required          ← Email thread / context
        └── for S0 topics_.docx
```

When used: Source of truth. When new documents arrive from the team, drop them here and tell Claude to read and update the agent knowledge files.

agent/knowledge/ — What Claude Knows

This folder holds structured markdown documents — summaries and extractions from the raw Knowledge folder, organised by topic type.

agent/knowledge/architecture/po-text-flow-diagram.md

One liner: Shows HOW data physically flows from STXH/STXL to ZPO_LONGTEXT — the plumbing diagrams that Claude uses to trace issues end-to-end.

Contains:

Section	Content
System landscape	Single SAP system diagram — no Hub, no replication
Phase 1 — Migration flow	Step-by-step: SELECT STXH → READ_TEXT → DELETE → INSERT
Phase 2 — Dual write flow	ME21N save → SAVE_TEXT → BAdI → ZPO_LONGTEXT
TDNAME construction	Exact rules for building TDNAME for EKKO vs EKPO

Section	Content
Debugging decision tree	Text missing? Follow these steps in sequence

Example of what Claude can answer with this file: > “Why is my BAdI not finding any text when I call READ_TEXT?” → > Claude checks the TDNAME construction rule and spots the EKPO parsing error.

agent/knowledge/domain/po-longtext-domain.md

One liner: Explains WHY this project exists, WHAT the business problem is, and WHAT rules govern the solution — the domain expert knowledge.

Contains:

Section	Content
Why classic long text is difficult	Compression, no SQL read, BDC problem, performance
What changes / what does not	STXH/STXL unchanged — ZPO_LONGTEXT is parallel
Text types reference	All TDID values for EKKO and EKPO with descriptions
Migration rules	Full load vs delta, DELETE+INSERT pattern, text deletion
Dual write triggers	Which BAdI method fires for which user action
SO parallel	How same pattern applies to VBAK/VBAP (Sales Orders)
Known constraints	132 char line limit, language handling, archive risk

Example of what Claude can answer with this file: > “Should we also migrate SO long texts?” → > Claude explains the VBAK/VBAP parallel pattern from domain knowledge, > not by guessing.

agent/knowledge/product/ — Currently Empty

One liner: Future home for formal product specs, SDD final versions, and roadmap decisions received from the product owner.

When to populate: When the SDD is finalised or new product decisions are communicated, place a .md summary here. Claude will use it to align all answers with the official product direction.

agent/codebase-index/ — How to Work in the Codebase

This folder holds operational reference documents — the fast-lookup material Claude uses when working on actual implementation tasks.

agent/codebase-index/codebase-overview.md

One liner: A fast index of every technical object in the skill — what exists, what it does, and how layers connect.

Contains:

Section	Content
Project overview	Name, type, domain in 3 lines
Tech stack	ABAP, BAdI, STXH/STXL, READ_TEXT
Key objects table	ZPO_LONGTEXT, ZCL_PO_LONGTEXT_HANDLER, ME_PROCESS_PO_CUST etc.
Architecture layers	ERP source → migration/BAdI → persistence → BDC/reporting
Critical constraints	3 rules that must always be respected

Example of what Claude can answer with this file: > “What class handles the dual-write?” → > Claude immediately says ZCL_PO_TEXT_BADI_IMPL calls ZCL_PO_LONGTEXT_HANDLER > without reading every other file.

agent/codebase-index/business-rules.md

One liner: The 7 rules Claude must NEVER violate when generating code or giving advice — the safety guardrails for the skill.

Contains:

Rule	What It Prevents
1. ZPO_LONGTEXT is parallel	Never replacing STXH/STXL accidentally
2. EBELP = ‘00000’ for EKKO	Missing header key field causing wrong results
3. TDNAME parsing for EKPO	Wrong READ_TEXT returning nothing
4. ORDER BY LINE_COUNTER	Text rendering in wrong order
5. DELETE + INSERT (not UPSERT)	Orphan lines when text is shortened
6. BAdI fires AFTER SAVE_TEXT	READ_TEXT returning empty before text committed
7. Test mode first for migration	Partial writes on production data

Example of what Claude can answer with this file: > Claude generates a migration loop — it automatically includes DELETE before INSERT, > adds ORDER BY LINE_COUNTER, and checks EBELP = ‘00000’ for header texts. > Without this file, these could be silently missed.

agent/codebase-index/data-flows.md

One liner: Step-by-step numbered flows for every major operation — Claude traces these when debugging or implementing.

Contains:

Flow	Steps
Flow 1 — Historic migration	SELECT STXH → parse TDNAME → READ_TEXT → DELETE → INSERT → COMMIT
Flow 2 — Dual write	ME21N save → SAVE_TEXT → BAdI fires → READ_TEXT → DELETE → INSERT
Flow 3 — BDC / Report read	SELECT * FROM zpo_longtext WHERE ebeln = ... ORDER BY line_counter

Also includes a key tables comparison:

Table	Written By	Read By	Notes
STXH	SAVE_TEXT (SAP std)	SE16, SELECT	Metadata only
STXL	SAVE_TEXT (SAP std)	READ_TEXT FM only	Compressed
ZPO_LONGTEXT	Migration + BAdI	SELECT, CDS, BDC	New flat table

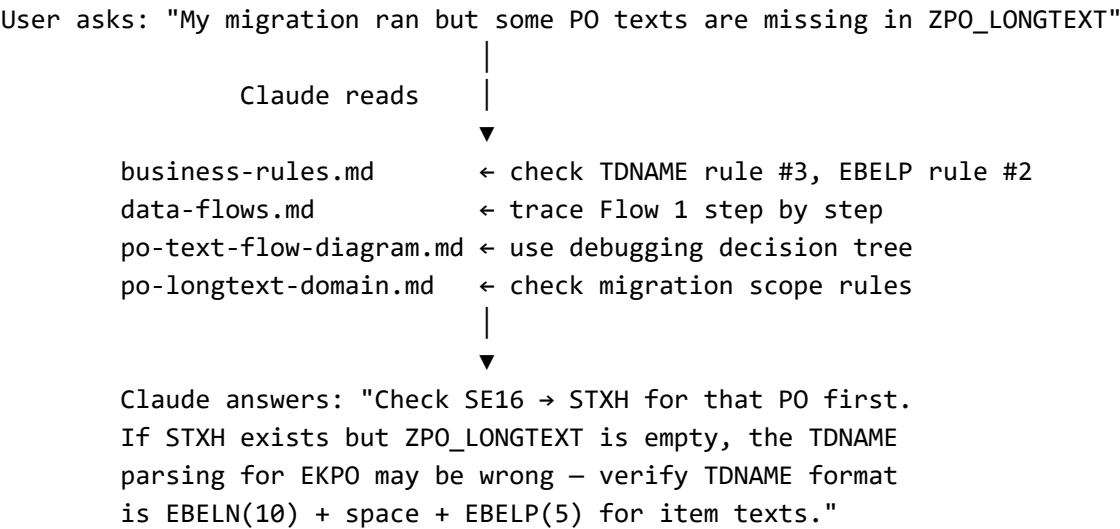
Example of what Claude can answer with this file: > “My BDC program needs to read PO notes” → > Claude immediately shows Flow 3 — SELECT from ZPO_LONGTEXT directly. > No READ_TEXT needed. No FM call. Plain SQL.

agent/artifacts/ — Currently Empty

One liner: Future version-controlled home for ABAP programs, migration scripts, and test outputs that Claude generates during implementation work.

When to populate: After Claude generates the migration report ZPO_LONGTEXT_MIGRATE code or BAdI implementation — save the final version here so it is tracked alongside the skill.

How All Files Work Together



Adding New Knowledge — Where to Put It

You Receive	Drop Here	Then
New SDD / design doc	Knowledge\PO Notes SD implemented...\	Tell Claude to read and update domain/product md files
Updated handover deck	Knowledge\PO Notes New Persistency\	Tell Claude to read and update architecture md files
Architecture change	agent\knowledge\architecture\	Edit po-text-flow-diagram.md directly
New business rule	agent\codebase-index\business-rules.md	Add Rule 8, 9 etc.
Generated ABAP code	agent\artifacts\	Save final version here

Always update repos.yaml when adding new raw documents to Knowledge\. **Always push to GitHub** after adding new documents.

Guide prepared: 2026-06-19 Skill: PO Long Text New Persistency — po-longtext-persistency-expert Repository: https://github.com/shivakums/PO_LongText_Persistencecy